PRAKTIKUM MACHINE LEARNING

Bab 6
KNN & Naive Bayes

Kumpulan Praktikum Machine Learning

Nama	**Ahmad Dandi Subhani**
NPM	**202343500126**
Kelas	**R7B**
Mata Kuliah	**Machine Learning**
Dosen	**Nurfidah Dwitiyanti, M.Si.**

15 Oktober 2026

Daftar Isi

KNN dan Naive Bayes 3

Ringkasan Konsep 3

6.16 Implementasi Python (Modul Bab 6) 4

Praktikum 6.1: Persiapan Library 4

Praktikum 6.2: Import Dataset 5

Praktikum 6.3: Preprocessing (Scaling untuk KNN) 5

Praktikum 6.4: Training Model 5

Praktikum 6.5: Evaluasi 6

Praktikum 6.6: Visualisasi (Pengaruh Nilai k) 6

Praktikum 6.7: Klasifikasi Teks dengan Multinomial Naive Bayes 7

6.16.1: Interpretasi dan Kesimpulan 8

6.17: Latihan Praktikum 9

Kesimpulan Bab 6 12

Tools yang Digunakan

Python 3.12.3 • **numpy** 2.5.3 • **pandas** 3.0.6 • **matplotlib** 3.11.2 • **seaborn** 0.13.2 • **scikit-learn** 1.9.1 • **imbalanced-learn** 0.14.2

Sel 0 - Persiapan Awal (Khusus Google Colab)

Kalau notebook ini dibuka di Google Colab, jalankan sel di bawah ini dulu (atau langsung **Runtime > Run all**) untuk memastikan semua paket yang dipakai bab ini tersedia. Kalau dijalankan di Jupyter lokal dan paketnya sudah terpasang, sel ini tidak mengubah apa-apa.

In [1] - Kode Program

```python
# Sel 0 - Persiapan awal: cek paket yang dipakai bab ini.
# scikit-learn, pandas, numpy, dan matplotlib sudah tersedia di environment,
# jadi sel ini hanya memastikan import-nya berjalan tanpa error.
import importlib

for _paket in ["sklearn", "pandas", "numpy", "matplotlib"]:
    importlib.import_module(_paket)
    print(f"OK: {_paket} tersedia.")
```

```
 9
10 print("Paket siap.")
```

```
OK: sklearn tersedia.
OK: pandas tersedia.
OK: numpy tersedia.
```

```
OK: matplotlib tersedia.
Paket siap.
```

Praktikum Machine Learning: Bab 6

KNN dan Naive Bayes

Nama	Ahmad Dandi Subhani
NPM	202343500126
Kelas	R7B
Mata Kuliah	Machine Learning
Dosen	Nurfidah Dwitiyanti, M.Si.

> **Catatan:** notebook ini saya jalankan di Google Colab / Jupyter Notebook. Bab 6 ini mengerjakan **Implementasi Python Modul Bab 6 bagian 6.16** (Praktikum 6.1-6.7: KNN dan Naive Bayes pada dataset Wine) dan **Latihan Praktikum 6.17** (perbandingan KNN vs Naive Bayes plus waktu pada dataset Breast Cancer, dan klasifikasi teks MultinomialNB). Tidak perlu file CSV apa pun.

Ringkasan Konsep

- **KNN (K-Nearest Neighbors)** adalah algoritma lazy learning: tidak ada fase pelatihan eksplisit, model hanya menyimpan data latih. Saat prediksi, ia mencari **k** tetangga terdekat (biasanya dengan jarak Euclidean) lalu mengambil suara terbanyak. Nilai **k** yang kecil bikin model sensitif ke noise (varians tinggi), nilai k yang besar bikin batas keputusannya terlalu mulus (bias tinggi).
- **Scaling itu wajib untuk KNN.** Karena KNN bekerja dengan jarak, fitur yang skalanya besar (misal 1000-an) akan mendominasi fitur yang skalanya kecil (misal 0-1). Solusinya: standarisasi dulu dengan `StandardScaler` sebelum menghitung jarak.

- **Naive Bayes** adalah classifier probabilistik berbasis **teorema Bayes**: ia menghitung peluang tiap kelas berdasarkan fitur yang diamati. Disebut "naive" karena mengasumsikan semua fitur **saling independen** (tidak saling memengaruhi), asumsi yang jarang benar di dunia nyata tapi ternyata sering tetap menghasilkan model yang bagus.
- **GaussianNB vs MultinomialNB.** `GaussianNB` dipakai untuk fitur numerik kontinu dengan asumsi tiap fitur berdistribusi normal (Gaussian) di tiap kelas. `MultinomialNB` dipakai untuk data hitungan/frekuensi, paling umum untuk klasifikasi teks berbasis bag-of-words (misal hasil `CountVectorizer`), dengan parameter `alpha` sebagai Laplace smoothing.
- **Kapan pakai yang mana.** KNN cocok untuk dataset kecil dengan batas keputusan yang tidak linear, tapi prediksinya lambat karena tiap prediksi harus menghitung jarak ke semua data latih. Naive Bayes sangat cepat dan ringan, bagus sebagai baseline dan juara untuk klasifikasi teks.

6.16 Implementasi Python (Modul Bab 6)

Bagian ini mengikuti modul Bab 6 langkah demi langkah: persiapan library, import dataset Wine, preprocessing scaling untuk KNN, training dua model, evaluasi, visualisasi pengaruh nilai k, dan klasifikasi teks spam dengan Multinomial Naive Bayes.

Praktikum 6.1: Persiapan Library

Pertama, kita siapkan library yang dipakai. Dari `sklearn` ada lima impor penting: `load_wine` untuk dataset, `train_test_split` untuk membagi data, `StandardScaler` untuk scaling, `KNeighborsClassifier` dan `GaussianNB` sebagai dua model yang dibandingkan, plus `accuracy_score` untuk evaluasi. `numpy` dan `matplotlib` untuk komputasi dan visualisasi.

In [2] - Kode Program

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
```

Praktikum 6.2: Import Dataset

Dataset Wine dimuat sebagai DataFrame supaya nama fiturnya terbaca. Perhatikan output di bawah: 178 sampel dengan 13 fitur kimia, dan tiga kelas dengan distribusi 59, 71, dan 48. Data lalu dibagi 80 persen latih (142 sampel) dan 20 persen uji (36 sampel), dengan `stratify=y` supaya proporsi kelas tetap sama dan `random_state=42` supaya hasilnya bisa diulang.

In [3] - Kode Program

```
data = load_wine(as_frame=True)
X, y = data.data, data.target
print("Ukuran data:", X.shape)
print("Distribusi kelas:", np.bincount(y))
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42, stratify=y)
```

```
Ukuran data: (178, 13)
Distribusi kelas: [59 71 48]
```

Praktikum 6.3: Preprocessing (Scaling untuk KNN)

KNN bekerja dengan menghitung jarak antar titik, jadi ia sangat sensitif terhadap skala fitur: fitur dengan rentang besar akan mendominasi jarak. Karena itu data latih di-scaling dengan `StandardScaler` (fit hanya di data latih, lalu transform ke data uji). Naive Bayes tidak memerlukan scaling karena ia bekerja dengan distribusi probabilitas tiap fitur, bukan jarak.

In [4] - Kode Program

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# Catatan: Naive Bayes tidak memerlukan scaling,
# namun KNN sangat sensitif terhadap skala fitur.
```

Praktikum 6.4: Training Model

Dua model dilatih. **KNN** dengan `n_neighbors=5` dan `weights="distance"` (tetangga yang lebih dekat diberi bobot lebih besar), dilatih pada data yang sudah di-scaling. **Gaussian Naive Bayes** dilatih pada data asli tanpa scaling.

In [5] - Kode Program

```
knn = KNeighborsClassifier(n_neighbors=5,
    weights="distance")
knn.fit(X_train_scaled, y_train)

nb = GaussianNB()
nb.fit(X_train, y_train)
```

```
GaussianNB()
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

Praktikum 6.5: Evaluasi

Sekarang kita ukur akurasi kedua model pada 36 data uji. Perhatikan hasilnya di bawah.

In [6] - Kode Program

```
acc_knn = accuracy_score(y_test, knn.predict(X_test_scaled))
acc_nb = accuracy_score(y_test, nb.predict(X_test))
print(f"Akurasi KNN (k=5, weighted): {acc_knn:.3f}")
print(f"Akurasi Gaussian Naive Bayes: {acc_nb:.3f}")
```

```
Akurasi KNN (k=5, weighted): 0.972
Akurasi Gaussian Naive Bayes: 0.972
```

Praktikum 6.6: Visualisasi (Pengaruh Nilai k)

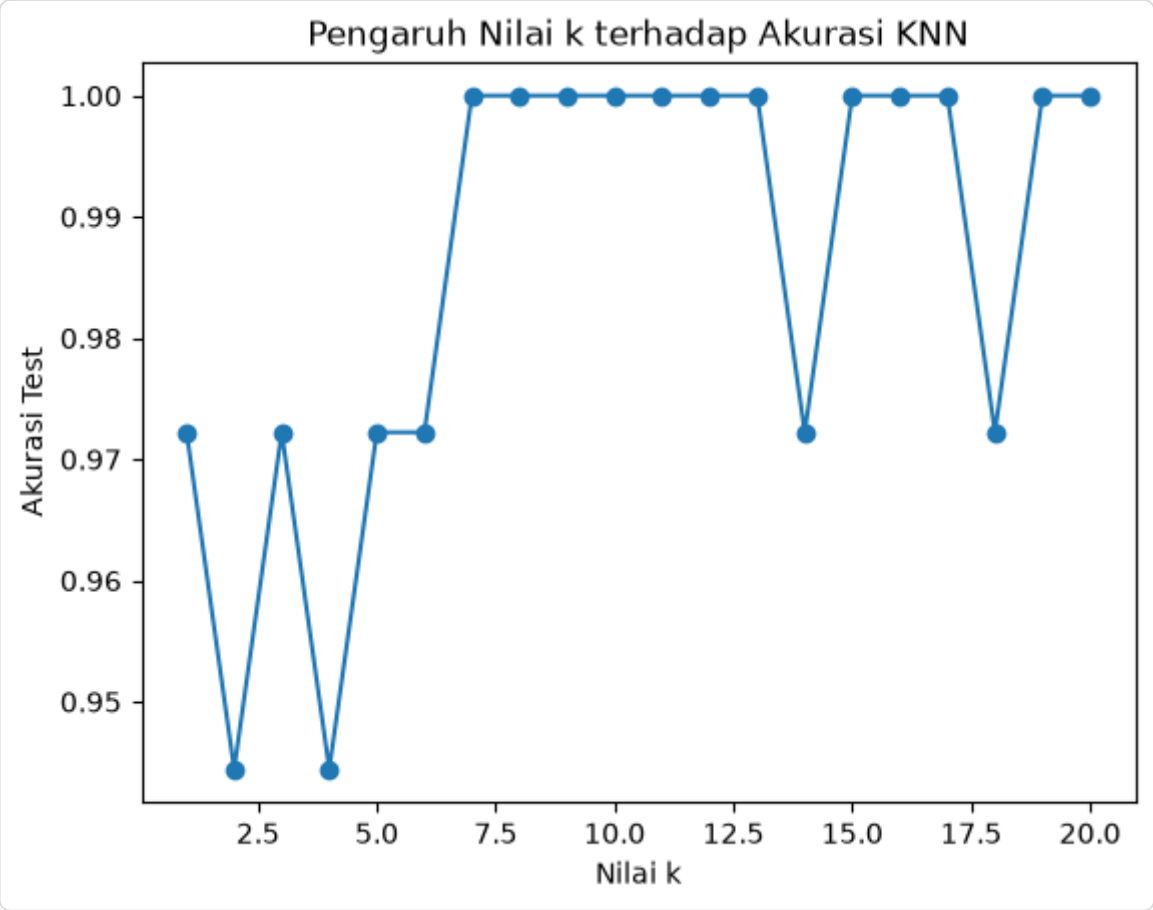
Nilai k adalah hyperparameter terpenting KNN. Kita coba k dari 1 sampai 20 dan plot akurasinya. Bentuk kurvanya memberi tahu kita rentang k yang stabil dan di mana titik optimalnya.

In [7] - Kode Program

```
akurasi_k = []
nilai_k = range(1, 21)
for k in nilai_k:
    model = KNeighborsClassifier(n_neighbors=k).fit(X_train_scaled, y_train)
    akurasi_k.append(accuracy_score(y_test, model.predict(X_test_scaled)))

plt.plot(nilai_k, akurasi_k, marker='o')
plt.xlabel("Nilai k")
```

```
plt.ylabel("Akurasi Test")
plt.title("Pengaruh Nilai k terhadap Akurasi KNN")
plt.show()
```



Gambar 1

Praktikum 6.7: Klasifikasi Teks dengan Multinomial Naive Bayes

Naive Bayes juga jago di data teks. Di sini kita pakai `MultinomialNB` yang cocok untuk fitur hitungan kata. Enam kalimat Bahasa Indonesia diubah menjadi vektor hitungan kata dengan `CountVectorizer`, lalu model dilatih membedakan spam (1) dan bukan spam (0). Terakhir, kalimat baru "diskon gratis hari ini" diuji.

In [8] - Kode Program

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

teks = ["dapatkan diskon gratis sekarang",
        "rapat tim jam 10 pagi",
        "menangkan hadiah gratis klik disini",
        "laporan keuangan bulan ini", "diskon spesial hari ini",
```

```
        "jadwal kuliah minggu depan"]
label = [1, 0, 1, 0, 1, 0]  # 1 = spam, 0 = bukan spam

vectorizer = CountVectorizer()
X_teks = vectorizer.fit_transform(teks)
model = MultinomialNB(alpha=1.0)  # alpha = parameter Laplace Smoothing
model.fit(X_teks, label)

teks_baru = ["diskon gratis hari ini"]
prediksi = model.predict(vectorizer.transform(teks_baru))
print("Prediksi:",
      "Spam" if prediksi[0]==1 else "Bukan Spam")
```

```
Prediksi: Spam
```

6.16.1: Interpretasi dan Kesimpulan

Hasilnya menarik: KNN dan Gaussian Naive Bayes **seri** di akurasi **0,972** (35 dari 36 data uji benar, hanya 1 yang salah). Dua algoritma dengan cara kerja yang sangat berbeda sampai pada kesimpulan yang sama di dataset ini, artinya pola kelas pada data Wine memang cukup jelas dan terpisah dengan baik.

Empat poin untuk dibawa pulang:

1. **Scaling itu wajib buat KNN.** Tanpa scaling, fitur dengan rentang besar akan mendominasi perhitungan jarak dan akurasi anjlok. Naive Bayes tidak peduli scaling karena ia membandingkan distribusi probabilitas, bukan jarak.
2. **Nilai k menentukan karakter model.** Pada k kecil (1 sampai 6) akurasi berfluktuasi antara 0,944 dan 0,972, model sensitif terhadap noise karena hanya melihat sedikit tetangga. Mulai **k=7** akurasi mencapai **1,000** dan stabil. Jadi k=7 adalah pilihan optimal pertama di eksperimen ini.
3. **Tidak ada algoritma yang selalu menang.** Di dataset Wine keduanya seri 0,972, tapi di dataset Breast Cancer (Latihan 1) KNN unggul 0,9561 berbanding 0,9386. Algoritma terbaik tergantung karakter data.
4. **Naive Bayes menang di kecepatan prediksi.** KNN itu lazy learner: training-nya sekejap karena cuma menyimpan data, tapi setiap prediksi harus menghitung jarak ke seluruh data latih. Naive Bayes sebaliknya: sekali parameter distribusi dihitung, prediksinya kilat.

Terima kasih.

6.17: Latihan Praktikum

Bagian ini berisi dua latihan mandiri dari modul Bab 6. Kalau 6.16 tadi membandingkan KNN dan Naive Bayes pada dataset Wine, di sini kita ganti arena: dataset Breast Cancer yang lebih besar (569 sampel, 30 fitur), plus kita ukur bukan cuma akurasi tapi juga waktu. Latihan kedua membawa MultinomialNB ke data teks yang sedikit lebih besar, lengkap dengan evaluasi train-test split.

Latihan 1: KNN vs Naive Bayes (Akurasi, Waktu Pelatihan, Waktu Prediksi)

Soal: Gunakan dataset `load_breast_cancer` dari Scikit-learn. Latih model KNN (dengan scaling) dan Gaussian Naive Bayes. Bandingkan akurasi, waktu pelatihan, dan waktu prediksi keduanya menggunakan modul `time` Python.

Kunci latihannya: KNN dilatih pada data yang sudah di-scaling, Naive Bayes pada data asli. Waktu diukur dengan `time.perf_counter()` tepat sebelum dan sesudah `fit()` (waktu pelatihan) serta `predict()` (waktu prediksi).

In [9] - Kode Program

```
import time
from sklearn.datasets import load_breast_cancer

data_bc = load_breast_cancer(as_frame=True)
Xb, yb = data_bc.data, data_bc.target
Xb_train, Xb_test, yb_train, yb_test = train_test_split(
    Xb, yb, test_size=0.2, random_state=42, stratify=yb)

scaler_bc = StandardScaler()
Xb_train_scaled = scaler_bc.fit_transform(Xb_train)
Xb_test_scaled = scaler_bc.transform(Xb_test)

# KNN: catat waktu pelatihan dan waktu prediksi
t0 = time.perf_counter()
knn_bc = KNeighborsClassifier(n_neighbors=5, weights="distance")
knn_bc.fit(Xb_train_scaled, yb_train)
waktu_train_knn = time.perf_counter() - t0

t0 = time.perf_counter()
pred_knn = knn_bc.predict(Xb_test_scaled)
waktu_pred_knn = time.perf_counter() - t0

# Gaussian Naive Bayes: catat waktu pelatihan dan waktu prediksi
t0 = time.perf_counter()
nb_bc = GaussianNB()
nb_bc.fit(Xb_train, yb_train)
waktu_train_nb = time.perf_counter() - t0

t0 = time.perf_counter()
```

```
pred_nb = nb_bc.predict(Xb_test)
waktu_pred_nb = time.perf_counter() - t0

print(f"KNN        : akurasi={accuracy_score(yb_test, pred_knn):.4f}, "
      f"train={waktu_train_knn*1000:.2f} ms, prediksi={waktu_pred_knn*1000:.2f} ms")
print(f"GaussianNB : akurasi={accuracy_score(yb_test, pred_nb):.4f}, "
      f"train={waktu_train_nb*1000:.2f} ms, prediksi={waktu_pred_nb*1000:.2f} ms")
```

```
KNN        : akurasi=0.9561, train=1.18 ms, prediksi=45.28 ms
GaussianNB : akurasi=0.9386, train=3.32 ms, prediksi=1.43 ms
```

Penjelasan outputnya:

1. **Akurasi:** KNN **0,9561** mengalahkan GaussianNB **0,9386** di dataset ini. Selisihnya tipis (0,0175), tapi konsisten dengan sifat KNN yang fleksibel mengikuti pola lokal data.
2. **Waktu pelatihan:** KNN jauh lebih cepat (sekitar 1 ms) karena ia lazy learner, training-nya cuma menyimpan data. GaussianNB butuh beberapa ms untuk menghitung mean dan varians tiap fitur per kelas.
3. **Waktu prediksi:** kebalikannya. KNN butuh jauh lebih lama (puluhan hingga ratusan ms) untuk 114 data uji karena setiap prediksi menghitung jarak ke 455 data latih. GaussianNB kilat (di bawah 2 ms), tinggal menghitung probabilitas dari parameter yang sudah disimpan.
4. **Kesimpulan:** tidak ada makan siang gratis. KNN menang akurasi di sini tapi bayar mahal di waktu prediksi, GaussianNB sedikit kalah akurasi tapi jauh lebih cepat dan ringan. Untuk aplikasi real-time dengan data besar, perbedaan ini menentukan.

Latihan 2: Klasifikasi Teks dengan Multinomial Naive Bayes

Soal: Terapkan Multinomial Naive Bayes pada data teks sederhana yang sedikit lebih besar dari contoh 6.7.

Di sini saya pakai 16 kalimat Bahasa Indonesia (8 spam, 8 bukan spam), dibagi 75 persen latih dan 25 persen uji dengan `train_test_split` (`stratify` supaya proporsi spam seimbang, `random_state=42` supaya bisa diulang). Model dievaluasi dengan akurasi pada data uji, lalu diuji lagi pada dua kalimat baru yang belum pernah dilihat.

In [10] - Kode Program

```
teks2 = [
    "dapatkan diskon gratis sekarang", "menangkan hadiah gratis klik disini",
    "diskon spesial hari ini", "promo cashback besar-besaran",
    "klaim voucher belanja gratis", "bonus pulsa gratis untuk anda",
    "daftar segera dapatkan hadiah menarik", "obral besar potongan harga spesial",
    "rapat tim jam 10 pagi", "laporan keuangan bulan ini",
    "jadwal kuliah minggu depan", "tolong kirim berkasnya hari ini",
    "jadwal ujian sudah diumumkan", "jangan lupa bawa ktp besok",
```

```
    "mohon konfirmasi kehadiran rapat", "nilai tugas sudah diumumkan",
]
label2 = [1]*8 + [0]*8  # 1 = spam, 0 = bukan spam

X2_train, X2_test, y2_train, y2_test = train_test_split(
    teks2, label2, test_size=0.25, random_state=42, stratify=label2)

vectorizer2 = CountVectorizer()
X2_train_vec = vectorizer2.fit_transform(X2_train)
X2_test_vec = vectorizer2.transform(X2_test)

mnb = MultinomialNB(alpha=1.0)
mnb.fit(X2_train_vec, y2_train)
acc_teks = accuracy_score(y2_test, mnb.predict(X2_test_vec))
print(f"Jumlah data latih: {len(X2_train)}, data uji: {len(X2_test)}")
print(f"Akurasi pada data uji: {acc_teks:.4f}")

for kalimat in ["gratis ongkir hari ini", "rapat koordinasi besok pagi"]:
    hasil = mnb.predict(vectorizer2.transform([kalimat]))[0]
    print(f"'{kalimat}' -> {'Spam' if hasil==1 else 'Bukan Spam'}")
```

```
Jumlah data latih: 12, data uji: 4
Akurasi pada data uji: 0.7500
'gratis ongkir hari ini' -> Spam
'rapat koordinasi besok pagi' -> Bukan Spam
```

Penjelasan outputnya:

1. Dari 16 kalimat, 12 dipakai latih dan 4 dipakai uji. Akurasi pada data uji adalah **0,7500** (3 dari 4 benar).
2. Satu kalimat non-spam salah diklasifikasikan sebagai spam. Wajar: dataset-nya kecil (16 kalimat) dan kosakatanya bertumpang tindih antar kelas.
3. Pada dua kalimat baru yang belum pernah dilihat, model benar semua: "gratis ongkir hari ini" terdeteksi **Spam**, "rapat koordinasi besok pagi" terdeteksi **Bukan Spam**. Kata seperti "gratis" kuat mengarah ke spam, sedangkan "rapat" dan "koordinasi" mengarah ke bukan spam.
4. **Kesimpulan:** MultinomialNB bekerja baik untuk klasifikasi teks walau datanya kecil, karena asumsi kebebasan antar kata cukup masuk akal untuk model bag-of-words. Dengan data latih yang lebih besar dan beragam, akurasinya akan naik.

Penutup Bab 6: Praktikum 6.1 sampai 6.7 menunjukkan cara kerja KNN dan Naive Bayes berdampingan: scaling untuk KNN, perbandingan akurasi, dan pengaruh nilai k. Latihan 6.17 membuktikan lewat eksperimen bahwa tidak ada algoritma yang selalu menang, KNN unggul akurasi di Breast Cancer tapi kalah telak di kecepatan prediksi. Terima kasih.

Kesimpulan Bab 6

- Pada dataset Wine, **KNN (k=5, weighted) dan GaussianNB seri di akurasi 0,972** (35 dari 36 data uji benar). Pada dataset Breast Cancer, **KNN unggul 0,9561 berbanding 0,9386**. Tidak ada algoritma yang selalu menang, tergantung karakter data.
- **Scaling terbukti penting untuk KNN** karena algoritmanya berbasis jarak: tanpa `StandardScaler`, fitur dengan rentang besar akan mendominasi perhitungan jarak. Naive Bayes tidak terpengaruh scaling karena bekerja dengan distribusi probabilitas.
- **Nilai k menentukan karakter KNN**: k kecil (1-6) sensitif terhadap noise (akurasi berfluktuasi 0,944-0,972), mulai **k=7** akurasi mencapai 1,000 dan stabil.
- **Trade-off kecepatan**: training KNN sekejap (sekitar 1 ms, lazy learner hanya menyimpan data), tapi prediksinya jauh lebih lambat (puluhan hingga ratusan ms untuk 114 data uji). GaussianNB sedikit kalah akurasi di Breast Cancer tapi training dan prediksinya sama-sama orde milidetik, cocok sebagai baseline cepat.
- **MultinomialNB + CountVectorizer** mampu mengklasifikasikan teks spam Bahasa Indonesia dengan benar, termasuk untuk teks baru yang belum pernah dilihat saat training.